

Sistemas Operativos

Práctica 2

Wenjie Chen - 100522255

Daniel Deblas Payo - 100522144

Xiya Ye - 100522159

3 de abril del 2025

Índice

Descripción del código.....	2
scripter.c.....	2
mygrep.c.....	4
Batería de pruebas.....	5
scripter.c.....	5
Caso de prueba 1.....	5
Caso de prueba 2.....	5
Caso de prueba 3.....	5
Caso de prueba 4.....	6
Caso de prueba 5.....	6
Caso de prueba 6.....	7
Caso de prueba 7.....	8
Caso de prueba 8.....	8
Caso de prueba 9.....	9
Caso de prueba 10.....	9
Caso de prueba 11.....	10
Caso de prueba 12.....	10
Caso de prueba 13.....	11
mygrep.c.....	12
Caso de prueba 1.....	12
Caso de prueba 2.....	12
Caso de prueba 3.....	12
Caso de prueba 4.....	13
Caso de prueba 5.....	13
Conclusión.....	14

Descripción del código

scripter.c

El programa *scripter.c* nos permite ejecutar varios comandos, éstos comandos se guardan en un archivo que se le pasa como argumento de entrada a *scripter.c*, por lo que tiene la siguiente interfaz: *./scripter <fichero de comandos>*.

Para la implementación de este programa, primero hemos declarado las variables que necesitamos. Después, nos aseguramos de que haya 2 argumentos de entrada. Una vez hecho esto, debemos leer la primera línea y verificar que sea igual que “## Script de SSOO”. Para eso abrimos el fichero con la función *open* y le decimos que nos lea con *read* justo el número de caracteres que necesitamos, y con eso lo comparamos con *strcmp* a lo que tiene que ser la primera línea, dependiendo de si es igual o no, terminamos el programa o seguimos leyendo el resto del archivo.

Para la lectura de los comandos usamos un bucle *while* leyendo de carácter en carácter, y cada vez que lea un salto de línea ‘\n’, significa que ha leído una línea, por lo que le insertamos ‘\0’ al final del string. Hecho esto, tenemos que comprobar que la línea no esté vacía, por lo que comparamos su longitud a 0, y si es así es que la línea solo tiene un salto de línea, por lo que terminamos la ejecución. En caso contrario, usamos la función de *procesar_linea* que hemos declarado, ésta se encarga de separar los comandos si es que hay varios (cuando hay pipes puede haber más de uno), y nos indica también si hay redirecciones o si se ejecuta en background o no. Cómo nos devuelve el número de comandos, lo que hacemos es dividir los posibles casos que podemos tener en dos, la primera es que se trate de un comando simple y en el segundo si contiene la línea tuberías.

Cuando se trata de un comando simple, lo que hacemos es crear un proceso hijo con *fork* y a éste le indicamos que nos ejecute el comando. Para eso, primero dividimos el comando usando la función *tokenizar_linea*, que lo que hace es separarnos el comando por espacios, por lo que así tenemos ya el comando y sus argumentos. Posteriormente, llamamos a la función *procesar_redirecciones* para que nos indique si esta línea tiene alguna redirección. Una vez tenemos esto, tenemos que abrir un archivo con el nombre que marque el comando, y cerramos el descriptor de entrada, salida o de errores estándar, dependiendo del caso, y usamos *dup* para que la estándar sea el archivo que hemos abierto. Hecho esto, podemos entonces ejecutar el comando con *execvp*. Por otra parte, el padre lo que debe hacer es comprobar si el comando se ejecuta en background o no, y así esperar a que el proceso hijo termine o no.



Cuando el número de comandos es mayor que 1 en una línea significa que estamos tratando con pipes. Para ello, vamos a usar un bucle *for* que nos va a iterar las veces que comandos haya, y así crear un proceso hijo por cada comando. Entonces, dentro de este bucle, creamos el pipe antes del proceso hijo, y como solo hace falta crear número de comandos - 1 pipes, le indicamos que cree pipe solo cuando no se trate del último comando, entonces creamos una tubería con *pipe*. Después creamos un proceso hijo con *fork*, y en este caso, el hijo hace lo mismo que en el caso de los comandos simples, dividir el comando, procesar las redirecciones y comprobar las redirecciones, un pequeño cambio que hay es que en las redirecciones de entrada y salida, solo lo comprobamos cuando se trate del primer o último comando, ya que en la mitad no tendría sentido. Por otra parte, tenemos que tener en cuenta que tienen que escribir en la tubería y leer de ella. Para ello, hemos dividido en dos casos, la primera es cuando se tratan de comandos que no sean la primera, en este caso, deben cerrar el descriptor de entrada estándar (*STDIN_FILENO*) y leer de la tubería, que hemos declarado como *p10*, ésta es la salida del comando anterior. En el segundo caso, todos los comandos menos la última, deben cerrar el descriptor de salida estándar (*STDOUT_FILENO*) y usar la salida del pipe, por lo que escribe en la tubería. Una vez que nos hemos encargado de las redirecciones de tuberías, inicializamos a NULL las variables y ejecutamos igual que en los comandos simples. Por otra parte, el padre debe cerrar la salida de la tubería y asignarle la lectura a la variable *p10* para todos los comandos que no sean el último, así los comandos pueden leer bien de la tubería. Además, debe esperar al hijo dependiendo si se ejecuta en background o no. Hecho esto, debemos usar de nuevo *waitpid* por si ha quedado algún hijo en ejecución, así evitamos el problema de proceso zombie.

Cuando se ha ejecutado una línea inicializamos el valor de background a 0 e iteramos nuestra variable para el bucle. Una vez hayan pasado todas las líneas, cerramos el archivo de comandos y terminamos el programa.

Para la función de *procesar_redirecciones* hemos tenido que hacer una modificación, ya que hemos tenido problemas con que se asignara el valor nulo, así que hemos implementado un bucle para ir reescribiendo la parte lo que haya detrás del símbolo de redirección, y tras hacer eso volver a comprobar la misma posición, ya que hemos reescrito.

mygrep.c

El programa *mygrep.c* tiene como objetivo mostrar por pantalla las líneas en las que aparece una cadena de texto dentro de un fichero. Para su funcionamiento, este programa debe recibir la ruta de un fichero y la cadena de texto que el usuario desea buscar.

Para implementar este programa, en primer lugar, se comprueba que el número de argumentos que recibe la función sea correcto. En este caso, debe ser 3 ya que como se ha mencionado anteriormente recibe un fichero, la cadena de texto y el propio programa.

En caso de que sea correcto, se abre el archivo en modo lectura y se comprueba que se ha abierto correctamente para proceder a la lectura del fichero. Antes de proceder a la lectura del archivo, se definen las variables necesarias para ello y para poder escribir la o las posibles líneas en las que se encuentra la cadena de texto.

Para leer el fichero, usamos la función `read` y un bucle `while` con lo que conseguimos leer 1024 bytes que se almacenan en el buffer. Mientras tanto, también recorremos el buffer carácter por carácter hasta llegar a un salto de línea, en donde se agrega `\0` para que sea una línea válida.

Tras ello, se verifica si la línea contiene la cadena buscada. En caso afirmativo se escribe la línea en la salida estándar. Además, en caso de que sea la primera vez que se ha encontrado la cadena, se marca como encontrada. Tras ello, se vuelve a inicializar la variable *pos* para la nueva línea.

Cuando nos encontramos en la última posición del array, significa que no queda espacio en el array, por lo que necesitamos expandir el array. Para ello usamos la función *realloc*, aumentando el *buffer_size* por dos, aquí, vamos a usar un array temporal, para no asignarle directamente el nuevo espacio al original, por si hay problemas de sobrescritura. Entonces, al array temporal le asignamos el espacio calculado, así, podemos expandir el espacio al array original.

Por último, en caso de que la cadena no se encuentre en el fichero se muestra un mensaje por pantalla en donde se le informa al usuario. Finalmente se cierra el archivo y finaliza el programa.

Batería de pruebas

scripter.c

Caso de prueba 1

- Entrada: ./scripter
- Descripción de prueba: llamar a la función sin argumentos suficientes
- Resultado:

```
a0522255@guernika-gpu:~$ ./scripter
Usage: ./scripter <ruta fichero>
```

Caso de prueba 2

- Entrada: ./scripter comandos.txt

Comandos.txt

```
ls -l
```

- Descripción de prueba: llamar a un archivo txt cuyo contenido no empieza como debe
- Resultado:

```
a0522255@guernika-gpu:~$ ./scripter comandos.txt
ERROR: fichero debe empezar por ## Script de SS00
```

Caso de prueba 3

- Entrada: ./scripter comandos
- Descripción de prueba: llamar a un archivo que no existe
- Resultado:

```
a0522255@guernika-gpu:~$ ./scripter comandos
open:: No such file or directory
```

Caso de prueba 4

- Entrada: ./scripter comandos.txt

comandos.txt

```
## Script de SS00
```

- Descripción de prueba: llamar a un archivo txt que tenga una línea vacía a parte de ## Script de SS00
- Resultado:

```
a0522255@guernika-gpu:~$ ./scripter comandos.txt  
línea en blanco
```

Caso de prueba 5

- Entrada: ./scripter comandos.txt

comandos.txt

```
## Script de SS00  
ls -l  
wc texto.txt
```

- Descripción de prueba: ejecución de comandos simples
- Resultado:

```
a0522255@guernika-gpu:~$ ./scripter comandos.txt  
total 120  
-rw-r--r-- 1 a0522255 alumnos 38 abr 2 16:18 comandos.txt  
drwxr-xr-x 2 a0522255 alumnos 0 ene 30 18:17 Descargas  
drwxr-xr-x 2 a0522255 alumnos 0 ene 30 18:17 Documentos  
drwxr-xr-x 2 a0522255 alumnos 0 ene 30 18:17 Escritorio  
drwxr-xr-x 2 a0522255 alumnos 0 ene 30 18:17 Imágenes  
drwxr-xr-x 2 a0522255 alumnos 0 ene 30 17:35 Maildir  
-rw-r--r-- 1 a0522255 alumnos 418 abr 2 14:27 makefile  
drwxr-xr-x 2 a0522255 alumnos 0 ene 30 18:17 Música  
-rw-r--r-- 1 a0522255 alumnos 16504 abr 2 14:45 mygrep  
-rw-r--r-- 1 a0522255 alumnos 2332 abr 2 14:23 mygrep.c  
-rw-r--r-- 1 a0522255 alumnos 2920 abr 2 14:45 mygrep.o  
drwxr-xr-x 2 a0522255 alumnos 0 ene 30 18:17 Plantillas  
drwxr-xr-x 2 a0522255 alumnos 0 ene 30 18:17 Público  
-rw-r--r-- 1 a0522255 alumnos 16992 abr 2 14:45 scripter  
-rw-r--r-- 1 a0522255 alumnos 11015 abr 2 14:22 scripter.c  
-rw-r--r-- 1 a0522255 alumnos 9296 abr 2 14:45 scripter.o  
drwxr-xr-x 2 a0522255 alumnos 0 ene 30 18:18 snap  
-rw-r--r-- 1 a0522255 alumnos 62 abr 2 16:11 texto.txt  
drwxr-xr-x 2 a0522255 alumnos 0 abr 2 14:20 thinclient_drives  
drwxr-xr-x 2 a0522255 alumnos 0 ene 30 18:17 Videos  
drwxr-xr-x 2 a0522255 alumnos 0 abr 1 15:28 windows  
5 15 62 texto.txt  
línea en blanco
```

Caso de prueba 6

- Entrada: ./scripter comandos.txt

comandos.txt

```
## Script de SS00  
ls -l  
cat texto.txt &
```

- Descripción de prueba: ejecución de comandos simples en background
- Resultado:

```
a0522255@guernika-gpu:~$ ./scripter comandos.txt  
total 120  
-rwx-----T 1 a0522255 alumnos 41 abr 2 16:37 comandos.txt  
drwx-----T 2 a0522255 alumnos 0 ene 30 18:17 Descargas  
drwx-----T 2 a0522255 alumnos 0 ene 30 18:17 Documentos  
drwx-----T 2 a0522255 alumnos 0 ene 30 18:17 Escritorio  
drwx-----T 2 a0522255 alumnos 0 ene 30 18:17 Imágenes  
drwx-----T 2 a0522255 alumnos 0 ene 30 17:35 Maildir  
-rwx-----T 1 a0522255 alumnos 418 abr 2 14:27 makefile  
drwx-----T 2 a0522255 alumnos 0 ene 30 18:17 Música  
-rwx--x--x 1 a0522255 alumnos 16504 abr 2 14:45 mygrep  
-rwx-----T 1 a0522255 alumnos 2332 abr 2 14:23 mygrep.c  
-rwx-----T 1 a0522255 alumnos 2920 abr 2 14:45 mygrep.o  
drwx-----T 2 a0522255 alumnos 0 ene 30 18:17 Plantillas  
drwx-----T 2 a0522255 alumnos 0 ene 30 18:17 Público  
-rwx--x--x 1 a0522255 alumnos 16992 abr 2 14:45 scripter  
-rwx-----T 1 a0522255 alumnos 11015 abr 2 14:22 scripter.c  
-rwx-----T 1 a0522255 alumnos 9296 abr 2 14:45 scripter.o  
drwx-----T 2 a0522255 alumnos 0 ene 30 18:18 snap  
-rwx-----T 1 a0522255 alumnos 62 abr 2 16:11 texto.txt  
drwx-----T 2 a0522255 alumnos 0 abr 2 14:20 thinclient_drives  
drwx-----T 2 a0522255 alumnos 0 ene 30 18:17 Vídeos  
drwx-----T 2 a0522255 alumnos 0 abr 1 15:28 windows  
línea en blanco  
a0522255@guernika-gpu:~$ Línea 1: fila 0  
Línea 2: fila 1  
Línea 3: fila 2  
nada  
fila 423
```


Caso de prueba 7

- Entrada: ./scripter comandos.txt

comandos.txt

```
## Script de SS00
./mygrep texto.txt Linea
```

- Descripción de prueba: ejecución del comando externo mygrep
- Resultado:

```
a0522255@guernika-gpu:~$ ./scripter comandos.txt
Linea 1: fila 0
Linea 2: fila 1
Linea 3: fila 2
linea en blanco
```

Caso de prueba 8

- Entrada: ./scripter comandos.txt

comandos.txt

```
## Script de SS00
cat texto.txt | ls -l
```

- Descripción de prueba: ejecución de comandos con pipes
- Resultado:

```
a0522255@guernika-gpu:~$ ./scripter comandos.txt
total 120
-rwx-----T 1 a0522255 alumnos 40 abr 2 16:54 comandos.txt
drwx-----T 2 a0522255 alumnos 0 ene 30 18:17 Descargas
drwx-----T 2 a0522255 alumnos 0 ene 30 18:17 Documentos
drwx-----T 2 a0522255 alumnos 0 ene 30 18:17 Escritorio
drwx-----T 2 a0522255 alumnos 0 ene 30 18:17 Imágenes
drwx-----T 2 a0522255 alumnos 0 ene 30 17:35 Maildir
-rwx-----T 1 a0522255 alumnos 418 abr 2 14:27 makefile
drwx-----T 2 a0522255 alumnos 0 ene 30 18:17 Música
-rwx--x--x 1 a0522255 alumnos 16504 abr 2 14:45 mygrep
-rwx-----T 1 a0522255 alumnos 2332 abr 2 14:23 mygrep.c
-rwx-----T 1 a0522255 alumnos 2920 abr 2 14:45 mygrep.o
drwx-----T 2 a0522255 alumnos 0 ene 30 18:17 Plantillas
drwx-----T 2 a0522255 alumnos 0 ene 30 18:17 Público
-rwx--x--x 1 a0522255 alumnos 16992 abr 2 14:45 scripter
-rwx-----T 1 a0522255 alumnos 11015 abr 2 14:22 scripter.c
-rwx-----T 1 a0522255 alumnos 9296 abr 2 14:45 scripter.o
drwx-----T 2 a0522255 alumnos 0 ene 30 18:18 snap
-rwx-----T 1 a0522255 alumnos 62 abr 2 16:11 texto.txt
drwx-----T 2 a0522255 alumnos 0 abr 2 14:20 thinclient_drives
drwx-----T 2 a0522255 alumnos 0 ene 30 18:17 Vídeos
drwx-----T 2 a0522255 alumnos 0 abr 1 15:28 windows
```

Caso de prueba 9

- Entrada: ./scripter comandos.txt

comandos.txt

```
## Script de SS00
cat < texto.txt
cat texto.txt > fich2 &
ls no_existe !> error.txt
```

fich2

```
holaaaaaa
```

texto.txt

```
Linea 1: fila 0
Linea 2: fila 1
Linea 3: fila 2
nada
fila 432
```

- Descripción de prueba: ejecución de comandos simples, secuencias con redirecciones y background
- Resultado:

```
a0522255@guernika-gpu:~$ ./scripter comandos.txt
Linea 1: fila 0
Linea 2: fila 1
Linea 3: fila 2
nada
fila 423
a0522255@guernika-gpu:~$ cat fich2
Linea 1: fila 0
Linea 2: fila 1
Linea 3: fila 2
nada
fila 423
```

Caso de prueba 10

- Entrada: ./scripter comandos.txt

comandos.txt contiene cat fich | sort | wc

- Descripción de prueba: ejecución de 3 comandos con pipes.
- Resultado:

```
a0522159@guernika:~/ssoo/p2$ cat fich | sort | wc
   5      5     29
a0522159@guernika:~/ssoo/p2$ ./scripter comandos.txt
   5      5     29
```

Caso de prueba 11

- Entrada: ./scripter comandos.txt
comandos.txt contiene ls | cat | sort | cat | wc
- Descripción de prueba: ejecución de más de 3 comandos con pipes.
- Resultado:

```
a0522159@guernika:~/ssoo/p2/final$ ls | cat | sort | cat | wc
      9      9      83
a0522159@guernika:~/ssoo/p2/final$ ./scripter comandos.txt
      9      9      83
```

Caso de prueba 12

- Entrada: ./scripter comandos.txt
- Descripción de prueba: ejecución de comandos con más de una redirección.
- Resultado:

```
a0522159@guernika:~/ssoo/p2/final$ cat comandos.txt
## Script de SS00
ls -l > fichero
sort < fichero > fich_sal
```

```
a0522159@guernika:~/ssoo/p2/final$ ./scripter comandos.txt
a0522159@guernika:~/ssoo/p2/final$ cat fichero
total 136
-rwx-----T 1 a0522159 alumnos 60 abr 3 20:06 comandos.txt
-rwx-----T 1 a0522159 alumnos 23 abr 3 19:53 fich
-rwx-----T 1 a0522159 alumnos 0 abr 3 20:07 fichero
-rwx-----T 1 a0522159 alumnos 57 abr 3 20:05 fich_errors
-rwx-----T 1 a0522159 alumnos 710 abr 3 20:07 fich_sal
-rwx-----T 1 a0522159 alumnos 418 abr 3 19:52 Makefile
-rwx--x--x 1 a0522159 alumnos 16504 abr 3 19:52 mygrep
-rwx-----T 1 a0522159 alumnos 2332 abr 3 19:51 mygrep.c
-rwx-----T 1 a0522159 alumnos 2920 abr 3 19:52 mygrep.o
-rwx--x--x 1 a0522159 alumnos 17144 abr 3 19:52 scripter
-rwx-----T 1 a0522159 alumnos 11730 abr 3 19:52 scripter.c
-rwx-----T 1 a0522159 alumnos 9888 abr 3 19:52 scripter.o
a0522159@guernika:~/ssoo/p2/final$ cat fich_sal
-rwx-----T 1 a0522159 alumnos 0 abr 3 20:07 fichero
-rwx-----T 1 a0522159 alumnos 23 abr 3 19:53 fich
-rwx-----T 1 a0522159 alumnos 57 abr 3 20:05 fich_errors
-rwx-----T 1 a0522159 alumnos 60 abr 3 20:06 comandos.txt
-rwx-----T 1 a0522159 alumnos 418 abr 3 19:52 Makefile
-rwx-----T 1 a0522159 alumnos 710 abr 3 20:07 fich_sal
-rwx-----T 1 a0522159 alumnos 2332 abr 3 19:51 mygrep.c
-rwx-----T 1 a0522159 alumnos 2920 abr 3 19:52 mygrep.o
-rwx-----T 1 a0522159 alumnos 9888 abr 3 19:52 scripter.o
-rwx-----T 1 a0522159 alumnos 11730 abr 3 19:52 scripter.c
-rwx--x--x 1 a0522159 alumnos 16504 abr 3 19:52 mygrep
-rwx--x--x 1 a0522159 alumnos 17144 abr 3 19:52 scripter
total 136
```

Caso de prueba 13

- Entrada: ./scripter comandos.txt

```
a0522159@guernika:~/ssoo/p2/final$ cat comandos.txt
## Script de SS00
find / -name '*.txt' &
ls -l
cat fich
```

- Descripción de prueba: comprobamos que se ejecuta correctamente cuando ejecutamos un comando de larga duración en background y después algunos simples, por lo que debería hacer primero las simples.
- Resultado:

```
a0522159@guernika:~/ssoo/p2/final$ ./scripter comandos.txt
total 144
-rwx-----T 1 a0522159 alumnos 56 abr 3 20:14 comandos.txt
-rwx-----T 1 a0522159 alumnos 23 abr 3 19:53 fich
-rwx-----T 1 a0522159 alumnos 710 abr 3 20:07 fichero
-rwx-----T 1 a0522159 alumnos 57 abr 3 20:05 fich_errors
-rwx-----T 1 a0522159 alumnos 710 abr 3 20:07 fich_sal
-rwx-----T 1 a0522159 alumnos 418 abr 3 19:52 Makefile
-rwx--x--x 1 a0522159 alumnos 16504 abr 3 19:52 mygrep
-rwx-----T 1 a0522159 alumnos 2332 abr 3 19:51 mygrep.c
-rwx-----T 1 a0522159 alumnos 2920 abr 3 19:52 mygrep.o
-rwx--x--x 1 a0522159 alumnos 17144 abr 3 19:52 scripter
-rwx-----T 1 a0522159 alumnos 11730 abr 3 19:52 scripter.c
-rwx-----T 1 a0522159 alumnos 9888 abr 3 19:52 scripter.o
palabra
oso
pato
gatos
1428744a0522159@guernika:~/ssoo/p2/final$ find: '/opt/mooney/vs-hamlet/examples': Permiso denegado
find: '/opt/prodigy/domains/incomplete': Permiso denegado
find: '/run/lightdm': Permiso denegado
find: '/run/wpa_supplicant': Permiso denegado
find: '/run/udisks2': Permiso denegado
find: '/run/libvirt/qemu/dbus': Permiso denegado
find: '/run/libvirt/nwfilter-binding': Permiso denegado
find: '/run/libvirt/nwfilter': Permiso denegado
find: '/run/libvirt/nodedev': Permiso denegado
find: '/run/libvirt/secrets': Permiso denegado
find: '/run/libvirt/interface': Permiso denegado
```

mygrep.c

Caso de prueba 1

- Entrada:
 - ./mygrep
 - ./mygrep texto.txt
- Descripción de prueba: llamar a la función sin argumento suficiente
- Resultado:

```
a0522255@guernika-gpu:~$ ./mygrep
Usage: ./mygrep <ruta_fichero> <cadena_busqueda>
```

```
a0522255@guernika-gpu:~$ ./mygrep texto.txt
Usage: ./mygrep <ruta_fichero> <cadena_busqueda>
```

Caso de prueba 2

- Entrada: ./mygrep texto.txt hola

texto.txt

```
Prueba uno,
Prueba dos, con coma
Hola
```

- Descripción de prueba: que busque una palabra que no esté contenida en el archivo texto.txt
- Resultado:

```
a0522255@guernika-gpu:~$ ./mygrep texto.txt hola
hola not found.
```

Caso de prueba 3

- Entrada: ./ mygrep archivo_no_exite.txt hola
- Descripción de prueba: llamar a un archivo que no existe
- Resultado:

```
a0522255@guernika-gpu:~$ ./mygrep archivo_no_exite.txt hola
Error al abrir el archivo: No such file or directory
```

Caso de prueba 4

- Entrada: ./ mygrep texto.txt Linea

texto.txt

```
Linea 1: fila 0
Linea 2: fila 1
Linea 3: fila 2
nada
fila 423
```

- Descripción de prueba: llamar al archivo que contiene más de una línea esa palabra
- Resultado:

```
a0522255@guernika-gpu:~$ ./mygrep texto.txt Linea
Linea 1: fila 0
Linea 2: fila 1
Linea 3: fila 2
```

Caso de prueba 5

- Entrada: ./ mygrep texto.txt en
- Descripción de prueba: llamar al archivo que contiene un párrafo largo
- Resultado:

```
a0522255@guernika-gpu:~$ ./mygrep texto.txt en
Puede que la tarea que me he impuesto de escribir una historia completa del pueblo romano desde el comienzo mismo de su existencia me recompense por el trabajo invertido en ella, no lo sé con certeza, ni creo que pueda aventurarlo. Porque veo que esta es una práctica común y antiguamente establecida, cada nuevo escritor está siempre persuadido de que ni lograrán mayor certidumbre en las materias de su narración, ni superarán la rudeza de la antigüedad en la excelencia de su estilo. Aunque esto sea así, seguirá siendo una gran satisfacción para mí haber tenido mi parte también en investigar, hasta el máximo de mis capacidades, los anales de la nación más importante del mundo, con un interés más profundo; y si en tal conjunto de escritores mi propia reputación resulta ocultada, me consuelo con la fama y la grandeza de aquellos que eclipsen mi fama. El asunto, además, es uno que exige un inmenso trabajo. Se remonta a más de 700 años atrás y, después de un comienzo modesto y humilde, ha crecido a tal magnitud que empieza a ser abrumador por su grandeza. No me cabe duda, tampoco, que para la mayoría de mis lectores los primeros tiempos y los inmediatamente siguientes, tienen poco atractivo; Se apresurarán a estos tiempos modernos en los que el poderío de una nación principal es desgastado por el deterioro interno. Yo, en cambio, buscaré una mayor recompensa a mis trabajos en poder cerrar los ojos ante los males de que nuestra generación ha sido testigo durante tantos años; tanto tiempo, al menos, como estoy dedicando todo mi pensamiento a reproducir los claros registros, libre de toda la ansiedad que pueden perturbar el historiador de su época, aunque no le puedan deformar la verdad.
```

Conclusión

La realización de este trabajo nos ha servido para afianzar nuestros conocimientos sobre la gestión de procesos. Además, también hemos podido entender cuál es el funcionamiento que tiene un intérprete de comandos en Linux. Al hacer el programa `scripter.c`, hemos podido comprender como usar los procesos hijos correctamente. Además, gracias al desarrollo de la parte en la que puede haber numerosos comandos, también hemos podido aprender a usar los pipes de manera correcta. Por otro lado, el desarrollo del programa `mygrep` nos ha servido para volver a repasar nuestros conocimientos sobre los ficheros y su funcionamiento con el sistema operativo.

Las principales dificultades que nos hemos encontrado a la hora de desarrollar el trabajo han sido a la hora de crear el programa `scripter`. Principalmente, hemos tenido problemas a la hora de hacer un correcto manejo de procesos zombie, ya que no sabíamos cómo implementar el código para que el padre al final espere a todos los hijos, al final caímos en la cuenta de que podemos usar la señal `SIGCHLD` para resolver este problema. Por otro lado, también hemos encontrado ciertas dificultades a la hora de desarrollar la parte del programa que permite la ejecución de secuencias de comandos debido a la falta de experiencia trabajando con pipes, también a que hemos tenido que tener mucho cuidado de ir cerrando los descriptores, ya que teníamos que tener claro cuáles eran los que nos iban a servir y cuáles no. Otro problema que hemos tenido han sido las redirecciones, ya que la función que se nos proporcionaba no nos funcionaba con nuestra implementación, por lo que hemos tenido que modificarlo, en vez de asignarle el valor nulo al string, lo que tuvimos que hacer fue pasar todos los caracteres hacia delante, por lo que tuvimos que reescribirlo.

En cuanto al desarrollo del comando `mygrep`, la principal dificultad ha sido dada por la gestión de memoria del buffer cuando este estaba lleno, para eso hemos tenido que usar la función de `malloc` (para reservar el espacio) y `realloc` (para expandir el espacio).

Por último, en la realización de la batería de pruebas hemos trabajado con el entorno y hemos podido probar que nuestros programas cumpliesen con los requisitos establecidos para el proyecto.